

Seance 1

Classes, Encapsulation et RAI
Les fondations d'un code C++ propre

Cours de renforcement C++

Table des matières

1	Introduction : Pourquoi les classes ?	3
1.1	Le probleme avec le C	3
1.2	La solution C++ : Les classes et RAI	3
2	Les classes : Les briques de base	5
2.1	Une classe, c'est quoi concretement ?	5
2.2	Private vs Public : L'encapsulation	5
3	Les constructeurs : Bien naitre	7
3.1	Le constructeur par default	7
3.2	Le constructeur avec parametres	7
3.3	Le constructeur de copie	8
4	Les destructeurs : Bien mourir	10
4.1	A quoi sert un destructeur ?	10
4.2	Ordre de destruction	10
5	RAI : Le concept qui change tout	12
5.1	Qu'est-ce que RAI exactement ?	12
5.2	Exemple 1 : Gestion de memoire	12
5.3	Exemple 2 : Gestion de fichiers	13
5.4	Exemple 3 : Mesure de temps	13
6	Const-correctness : Promettre de ne pas modifier	15
6.1	Les methodes const	15
6.2	Parametres const	15
7	Exercices pratiques	17
7.1	Exercice 1 : Classe Compteur (facile)	17
7.2	Exercice 2 : Classe Array avec RAI (moyen)	17
7.3	Exercice 3 : Timer RAI (moyen)	19
7.4	Exercice 4 : FileWriter RAI (difficile)	20
8	Pieges courants et bonnes pratiques	22
8.1	Piege 1 : Oublier le destructeur virtuel	22
8.2	Piege 2 : La copie superficielle	23
8.3	Bonne pratique : La regle du zero ou la regle de cinq	24

9 Quiz de validation	25
9.1 Questions	25
9.2 Reponses	25
10 Pour aller plus loin	26
10.1 Exercice bonus : Smart Logger	26
10.2 Exercice bonus : Mutex Guard	26
10.3 Ressources supplementaires	26

1 Introduction : Pourquoi les classes ?

1.1 Le probleme avec le C

Imaginez que vous devez gerer un fichier dans votre programme. En C, vous ecririez probablement quelque chose comme ca :

```
1 FILE* fichier = fopen("data.txt", "r");
2 if (fichier == NULL) {
3     printf("Erreur !\n");
4     return -1;
5 }
6
7 // ... utilisation du fichier ...
8
9 fclose(fichier); // N'oubliez SURTOUT PAS cette ligne !
```

Le probleme

Et si vous oubliez le `fclose()` ? Le fichier reste ouvert, la memoire fuite, et votre programme peut crasher apres avoir ouvert trop de fichiers !

Et si une erreur se produit entre `fopen()` et `fclose()` ? Vous sortez de la fonction sans fermer le fichier !

1.2 La solution C++ : Les classes et RAI

En C++, on peut faire beaucoup mieux. On va **emballer** toute la logique d'ouverture/fermeture dans une classe :

```
1 class Fichier {
2 private:
3     FILE* handle;
4 public:
5     // Constructeur : s'execute automatiquement a la creation
6     Fichier(const std::string& nom) {
7         handle = fopen(nom.c_str(), "r");
8         if (!handle) {
9             throw std::runtime_error("Impossible d'ouvrir le fichier");
10        }
11        std::cout << "Fichier ouvert !\n";
12    }
13
14    // Destructeur : s'execute automatiquement a la destruction
15    ~Fichier() {
16        if (handle) {
17            fclose(handle);
18            std::cout << "Fichier ferme automatiquement !\n";
19        }
20    }
21
22    // Methode pour lire
23    std::string lire() {
24        // ... code de lecture ...
25    }
26};
```

La magie

Maintenant, utilisez votre classe comme ca :

```
1 {  
2     Fichier f("data.txt"); // Constructeur appele : fichier ouvert  
3     std::cout << f.lire();  
4     // ... utilisez f ...  
5 } // <- Destructeur appele automatiquement : fichier ferme !
```

Vous ne pouvez PLUS oublier de fermer le fichier! Le destructeur est appele automatiquement quand l'objet sort de la portee (les accolades).

C'est **ca**, le **RAII**! Resource Acquisition Is Initialization = "Quand tu crees l'objet, tu acquiers la ressource. Quand l'objet meurt, la ressource est liberee automatiquement."

2 Les classes : Les briques de base

2.1 Une classe, c'est quoi concretement ?

Pensez a une classe comme a un **plan de construction** (blueprint en anglais). Ce n'est PAS la maison elle-meme, c'est le plan pour construire des maisons.

```

1 // Ceci est le PLAN (la classe)
2 class Voiture {
3 private:
4     std::string marque;
5     int vitesse;
6
7 public:
8     // Constructeur : comment construire une voiture
9     Voiture(const std::string& m) : marque(m), vitesse(0) {}
10
11    // Methodes : ce qu'une voiture peut faire
12    void accelerer() {
13        vitesse += 10;
14    }
15
16    void afficher() {
17        std::cout << marque << " roule a " << vitesse << " km/h\n";
18    }
19 };
20
21 // Ici, on construit des voitures CONCRETES (des objets)
22 int main() {
23     Voiture maVoiture("Toyota"); // Construction d'une voiture
24     Voiture taVoiture("BMW");    // Construction d'une autre voiture
25
26     maVoiture.accelerer();
27     maVoiture.afficher(); // Toyota roule a 10 km/h
28
29     taVoiture.afficher(); // BMW roule a 0 km/h
30 }

```

Vocabulaire important

- **Classe** = Le plan, la recette, le modele
- **Objet** = Une instance concrete construite a partir du plan
- **Membre** = Une variable qui appartient a la classe (ex : vitesse, marque)
- **Methode** = Une fonction qui appartient a la classe (ex : accelerer)

2.2 Private vs Public : L'encapsulation

Question : Pourquoi mettre certaines choses en **private** ?

Reponse simple : Pour empecher qu'on fasse n'importe quoi avec !

```

1 class CompteBancaire {
2 private:
3     double solde; // PRIVE : personne ne peut y acceder directement
4
5 public:
6     CompteBancaire() : solde(0.0) {}

```

```
7
8 // Methode publique pour deposter (avec validation)
9 void deposter(double montant) {
10     if (montant > 0) {
11         solde += montant;
12     } else {
13         std::cout << "Montant invalide !\n";
14     }
15 }
16
17 // Methode publique pour consulter
18 double consulter() const {
19     return solde;
20 }
21 };
22
23 int main() {
24     CompteBancaire compte;
25
26     // compte.solde = 1000000; // ERREUR ! solde est prive
27
28     compte.deposer(100); // OK : passage par la methode publique
29     compte.deposer(-50); // Refuse : montant negatif
30
31     std::cout << compte.consulter() << " euros\n"; // 100 euros
32 }
```

Pourquoi c'est important ?

Sans private : N'importe qui pourrait écrire `compte.solde = -99999` et votre banque serait en faillite !

Avec private : Vous contrôlez TOUTES les modifications via vos méthodes. Vous pouvez ajouter des vérifications, des logs, etc.

C'est ça l'encapsulation : Cacher les détails internes et n'exposer qu'une interface propre et contrôlée.

3 Les constructeurs : Bien naitre

3.1 Le constructeur par défaut

Un constructeur, c'est une fonction speciale qui s'appelle automatiquement quand vous creez un objet.

```
1 class Point {
2 private:
3     int x, y;
4
5 public:
6     // Constructeur par défaut (sans parametres)
7     Point() {
8         x = 0;
9         y = 0;
10        std::cout << "Point cree a (0, 0)\n";
11    }
12
13    void afficher() {
14        std::cout << "(" << x << ", " << y << ")\n";
15    }
16 };
17
18 int main() {
19     Point p; // Appelle automatiquement Point()
20     p.afficher(); // (0, 0)
21 }
```

3.2 Le constructeur avec parametres

Souvent, on veut initialiser avec des valeurs specifiques :

```
1 class Point {
2 private:
3     int x, y;
4
5 public:
6     // Constructeur par défaut
7     Point() : x(0), y(0) {
8         std::cout << "Point par défaut\n";
9     }
10
11    // Constructeur avec parametres
12    Point(int px, int py) : x(px), y(py) {
13        std::cout << "Point cree a (" << x << ", " << y << ")\n";
14    }
15
16    void afficher() {
17        std::cout << "(" << x << ", " << y << ")\n";
18    }
19 };
20
21 int main() {
22     Point p1; // Appelle Point()
23     Point p2(10, 20); // Appelle Point(int, int)
24
25     p1.afficher(); // (0, 0)
```

```

26     p2.afficher(); // (10, 20)
27 }

```

La syntaxe avec deux-points

`Point(int px, int py) : x(px), y(py) {}`

Les deux-points : introduisent la **liste d'initialisation**. C'est la façon **preferable** d'initialiser les membres, car c'est plus efficace que de les affecter dans le corps du constructeur.

Equivalent moins efficace :

```

1 Point(int px, int py) {
2     x = px; // Affectation (moins bien)
3     y = py;
4 }

```

3.3 Le constructeur de copie

Que se passe-t-il quand on copie un objet ?

```

1 class Tableau {
2 private:
3     int* data;
4     size_t taille;
5
6 public:
7     // Constructeur normal
8     Tableau(size_t t) : taille(t) {
9         data = new int[taille];
10        std::cout << "Tableau de " << taille << " cree\n";
11    }
12
13    // Constructeur de copie
14    Tableau(const Tableau& autre) {
15        taille = autre.taille;
16        data = new int[taille]; // Nouvelle allocation !
17        for (size_t i = 0; i < taille; ++i) {
18            data[i] = autre.data[i]; // Copie element par element
19        }
20        std::cout << "Tableau copie\n";
21    }
22
23    // Destructeur
24    ~Tableau() {
25        delete[] data;
26        std::cout << "Tableau detruit\n";
27    }
28 };
29
30 int main() {
31     Tableau t1(5); // Constructeur normal
32     Tableau t2 = t1; // Constructeur de copie appele !
33     Tableau t3(t1); // Constructeur de copie aussi !
34 }

```

ATTENTION : La copie superficielle

Si vous n'écrivez PAS de constructeur de copie, C++ en genere un automatiquement qui copie betement tous les membres.

Probleme : Si vous avez un pointeur, les deux objets pointent vers la MEME memoire ! Quand l'un est detruit, il libere la memoire, et l'autre a un pointeur invalide (dangling pointer).

Solution : Ecrire votre propre constructeur de copie qui fait une **copie profonde** (deep copy).

4 Les destructeurs : Bien mourir

4.1 A quoi sert un destructeur ?

Le destructeur est une fonction speciale qui s'appelle automatiquement quand un objet est detruit. C'est LA ou vous liberez les ressources.

```

1  class Logger {
2  private:
3      std::ofstream fichier;
4
5  public:
6      Logger(const std::string& nom) {
7          fichier.open(nom);
8          std::cout << "Logger démarre\n";
9      }
10
11     ~Logger() {
12         if (fichier.is_open()) {
13             fichier.close();
14         }
15         std::cout << "Logger arrete\n";
16     }
17
18     void log(const std::string& message) {
19         fichier << message << std::endl;
20     }
21 };
22
23 void fonction() {
24     Logger log("debug.log");
25     log.log("Debut de la fonction");
26     log.log("Calculs en cours...");
27     // Pas besoin de fermer manuellement le fichier !
28 } // <- Le destructeur est appele automatiquement ici

```

4.2 Ordre de destruction

Les destructeurs sont appeles dans l'ordre INVERSE de la construction :

```

1  class A {
2  public:
3      A() { std::cout << "A construit\n"; }
4      ~A() { std::cout << "A detruit\n"; }
5  };
6
7  class B {
8  public:
9      B() { std::cout << "B construit\n"; }
10     ~B() { std::cout << "B detruit\n"; }
11 };
12
13 int main() {
14     std::cout << "Debut\n";
15     A a;
16     B b;
17     std::cout << "Fin\n";
18 }

```

```
19
20 // Affiche :
21 // Debut
22 // A construit
23 // B construit
24 // Fin
25 // B detruit <- Inverse !
26 // A detruit <- Inverse !
```

Pourquoi cet ordre ?

C'est logique : si B depend de A, il faut detruire B AVANT A. Sinon B essaierait d'utiliser A alors qu'il est deja detruit !

C'est comme un empilement d'assiettes : la derniere posee est la premiere enlevee (LIFO : Last In First Out).

5 RAI : Le concept qui change tout

5.1 Qu'est-ce que RAI exactement ?

RAI = Resource Acquisition Is Initialization

Traduit en français : “L’acquisition de la ressource SE FAIT dans l’initialisation (le constructeur)”

En langage simple :

1. Quand vous créez l’objet (constructeur), vous prenez la ressource (memoire, fichier, mutex, etc.)
2. Quand l’objet meurt (destructeur), vous libérez AUTOMATIQUEMENT la ressource
3. Impossible d’oublier de libérer, car le destructeur est TOUJOURS appelé

5.2 Exemple 1 : Gestion de memoire

Sans RAI (mauvais) :

```

1 void fonction() {
2     int* data = new int[1000];
3
4     // ... beaucoup de code ...
5
6     if (erreur) {
7         return; // OUPS ! On a oublie de delete[] data !
8     }
9
10    // ... encore du code ...
11
12    delete[] data; // Libere la memoire... si on arrive jusqu'ici
13 }
```

Avec RAI (bon) :

```

1 class Array {
2 private:
3     int* data;
4     size_t taille;
5
6 public:
7     Array(size_t t) : taille(t) {
8         data = new int[taille]; // Acquisition dans le constructeur
9     }
10
11     ~Array() {
12         delete[] data; // Liberation dans le destructeur
13     }
14
15     // Empêcher la copie pour simplifier
16     Array(const Array&) = delete;
17     Array& operator=(const Array&) = delete;
18 };
19
20 void fonction() {
21     Array arr(1000); // Memoire allouee
22
23     if (erreur) {
24         return; // Pas de probleme ! Le destructeur sera appele
25     }
```

```

26
27     // ... du code ...
28
29 } // Destructeur appele automatiquement : memoire liberee !

```

5.3 Exemple 2 : Gestion de fichiers

```

1  class FileHandler {
2  private:
3      FILE* file;
4      std::string filename;
5
6  public:
7      FileHandler(const std::string& name) : filename(name) {
8          file = fopen(name.c_str(), "r");
9          if (!file) {
10             throw std::runtime_error("Impossible d'ouvrir " + name);
11         }
12         std::cout << "Fichier " << name << " ouvert\n";
13     }
14
15     ~FileHandler() {
16         if (file) {
17             fclose(file);
18             std::cout << "Fichier " << filename << " ferme\n";
19         }
20     }
21
22     std::string lireLigne() {
23         char buffer[256];
24         if (fgets(buffer, sizeof(buffer), file)) {
25             return std::string(buffer);
26         }
27         return "";
28     }
29 };
30
31 void traiterFichier() {
32     FileHandler fh("data.txt"); // Fichier ouvert
33
34     std::string ligne = fh.lireLigne();
35
36     if (ligne.empty()) {
37         return; // Fichier ferme automatiquement !
38     }
39
40     // Traitement...
41
42 } // Fichier ferme automatiquement ici aussi !

```

5.4 Exemple 3 : Mesure de temps

Un exemple tres pratique : mesurer le temps d'execution d'une fonction.

```

1  class Timer {
2  private:
3      std::chrono::time_point<std::chrono::high_resolution_clock> start;

```

```
4     std::string nom;
5
6 public:
7     Timer(const std::string& n) : nom(n) {
8         start = std::chrono::high_resolution_clock::now();
9         std::cout << "[" << nom << "] Debut\n";
10    }
11
12    ~Timer() {
13        auto end = std::chrono::high_resolution_clock::now();
14        auto duree = std::chrono::duration_cast<std::chrono::
15            milliseconds>(
16            end - start
17        ).count();
18        std::cout << "[" << nom << "] Fin : " << duree << " ms\n";
19    }
20};
21
22 void calculComplexe() {
23     Timer t("Calcul"); // Demarre le chronometre
24
25     // Simule un calcul long
26     for (int i = 0; i < 1000000; ++i) {
27         // ...
28     }
29 } // Affiche automatiquement le temps ecoule !
30
31 int main() {
32     calculComplexe();
33     // Affiche : [Calcul] Debut
34     //           [Calcul] Fin : 15 ms
35 }
```

L'avantage du Timer RAI

- Vous ne pouvez pas oublier de stopper le chronometre
- Meme si une exception est levee, le temps est affiche
- Tres facile a utiliser : juste creer l'objet au debut de la fonction
- Pas de code de nettoyage explicite

6 Const-correctness : Promettre de ne pas modifier

6.1 Les methodes const

Quand vous mettez `const` apres une methode, vous promettez qu'elle ne modifie PAS l'objet.

```

1 class Point {
2 private:
3     int x, y;
4
5 public:
6     Point(int px, int py) : x(px), y(py) {}
7
8     // Cette methode ne modifie PAS l'objet
9     void afficher() const {
10         std::cout << "(" << x << ", " << y << ")\n";
11         // x = 10; // ERREUR ! On a promis de ne pas modifier !
12     }
13
14     // Cette methode MODIFIE l'objet
15     void deplacer(int dx, int dy) {
16         x += dx;
17         y += dy;
18     }
19
20     // Getter : retourne une valeur sans modifier l'objet
21     int getX() const { return x; }
22     int getY() const { return y; }
23 };
24
25 void fonction(const Point& p) {
26     p.afficher(); // OK : afficher() est const
27     // p.deplacer(5, 5); // ERREUR ! p est const, on ne peut pas le
28     // modifier
29     int valeur = p.getX(); // OK : getX() est const
30 }

```

Pourquoi c'est important ?

Sans const : Le compilateur ne peut pas verifier que vous ne modifiez pas accidentellement un objet.

Avec const : Le compilateur GARANTIT qu'une fonction ne modifiera pas l'objet. C'est comme un contrat.

Regle d'or : Mettez `const` partout ou vous ne modifiez pas l'objet. Le compilateur est votre ami !

6.2 Parametres const

```

1 class Personne {
2 private:
3     std::string nom;
4     int age;
5
6 public:
7     Personne(const std::string& n, int a) : nom(n), age(a) {}
8
9     // Passe par const reference : pas de copie, pas de modification

```

```
10 void saluer(const Personne& autre) const {
11     std::cout << nom << " dit bonjour a " << autre.nom << "\n";
12 }
13
14 std::string getNom() const { return nom; }
15 int getAge() const { return age; }
16 };
17
18 int main() {
19     Personne alice("Alice", 25);
20     Personne bob("Bob", 30);
21
22     alice.saluer(bob); // Alice dit bonjour a Bob
23 }
```

Passage par const reference

const std::string& n

Pourquoi & (reference) ? Pour eviter de copier (plus efficace).

Pourquoi const ? Pour promettre qu'on ne va pas modifier le parametre.

Alternative (mauvaise) : std::string n copie toute la chaine (lent et inutile).

7 Exercices pratiques

7.1 Exercice 1 : Classe Compteur (facile)

Objectif : Créer une classe qui compte et affiche automatiquement le résultat.

Enoncé

Créez une classe `Compteur` qui :

- A un membre privé `valeur` (int)
- Un constructeur qui initialise la valeur à 0
- Une méthode `incrementer()` qui augmente la valeur de 1
- Une méthode `afficher()` const qui affiche la valeur
- Un destructeur qui affiche "Compteur détruit : valeur finale = X"

Utilisation attendue :

```
1 int main() {
2     Compteur c;
3     c.incrementer();
4     c.incrementer();
5     c.afficher(); // Affiche : 2
6 } // Affiche : Compteur détruit : valeur finale = 2
```

Solution :

```
1 class Compteur {
2 private:
3     int valeur;
4
5 public:
6     Compteur() : valeur(0) {
7         std::cout << "Compteur cree\n";
8     }
9
10    ~Compteur() {
11        std::cout << "Compteur détruit : valeur finale = "
12                << valeur << "\n";
13    }
14
15    void incrementer() {
16        valeur++;
17    }
18
19    void afficher() const {
20        std::cout << "Valeur : " << valeur << "\n";
21    }
22 };
```

7.2 Exercice 2 : Classe Array avec RAI (moyen)

Objectif : Gérer un tableau dynamique avec allocation/libération automatique.

Enonce

Creez une classe Array qui :

- Prend une taille dans le constructeur
- Alloue un tableau d'entiers avec `new[]`
- A une methode `set(index, valeur)` pour modifier un element
- A une methode `get(index) const` pour lire un element
- Libere automatiquement la memoire dans le destructeur

Solution :

```

1 class Array {
2 private:
3     int* data;
4     size_t taille;
5
6 public:
7     // Constructeur : alloue la memoire
8     Array(size_t t) : taille(t) {
9         data = new int[taille];
10        std::cout << "Array de " << taille << " elements cree\n";
11
12        // Initialise a zero
13        for (size_t i = 0; i < taille; ++i) {
14            data[i] = 0;
15        }
16    }
17
18    // Destructeur : libere la memoire
19    ~Array() {
20        delete[] data;
21        std::cout << "Array detruit\n";
22    }
23
24    // Interdit la copie (pour simplifier)
25    Array(const Array&) = delete;
26    Array& operator=(const Array&) = delete;
27
28    void set(size_t index, int valeur) {
29        if (index < taille) {
30            data[index] = valeur;
31        }
32    }
33
34    int get(size_t index) const {
35        if (index < taille) {
36            return data[index];
37        }
38        return 0;
39    }
40
41    size_t size() const {
42        return taille;
43    }
44 };
45

```

```

46 int main() {
47     Array arr(5);
48     arr.set(0, 10);
49     arr.set(1, 20);
50
51     std::cout << "arr[0] = " << arr.get(0) << "\n"; // 10
52     std::cout << "arr[1] = " << arr.get(1) << "\n"; // 20
53
54     // Pas besoin de delete : c'est automatique !
55 }

```

7.3 Exercice 3 : Timer RAI (moyen)

Objectif : Créer un chronomètre qui mesure automatiquement le temps d'exécution.

Enoncé

Créez une classe `ScopedTimer` qui :

- Demarre le chronomètre dans le constructeur
- Arrête et affiche le temps écoulé dans le destructeur
- Affiche le temps en millisecondes

Indice : Utilisez `std::chrono`

Solution :

```

1  #include <chrono>
2
3  class ScopedTimer {
4  private:
5      std::chrono::time_point<std::chrono::high_resolution_clock> debut;
6      std::string nom;
7
8  public:
9      ScopedTimer(const std::string& n) : nom(n) {
10         debut = std::chrono::high_resolution_clock::now();
11     }
12
13     ~ScopedTimer() {
14         auto fin = std::chrono::high_resolution_clock::now();
15         auto duree = std::chrono::duration_cast<
16             std::chrono::milliseconds
17         >(fin - debut).count();
18
19         std::cout << "[" << nom << "] Temps : "
20             << duree << " ms\n";
21     }
22 };
23
24 void fonctionLente() {
25     ScopedTimer timer("fonctionLente");
26
27     // Simule un calcul long
28     int somme = 0;
29     for (int i = 0; i < 100000000; ++i) {
30         somme += i;
31     }

```

```

32
33 } // Affiche automatiquement le temps
34
35 int main() {
36     fonctionLente();
37 }

```

7.4 Exercice 4 : FileWriter RAI (difficile)

Objectif : Créer une classe qui gère l'ouverture/fermeture d'un fichier.

Enoncé

Créez une classe `FileWriter` qui :

- Ouvre un fichier en écriture dans le constructeur
- Lance une exception si l'ouverture échoue
- A une méthode `write()` pour écrire une ligne
- Ferme automatiquement le fichier dans le destructeur
- Affiche un message lors de l'ouverture et de la fermeture

Solution :

```

1  #include <fstream>
2
3  class FileWriter {
4  private:
5      std::ofstream fichier;
6      std::string nomFichier;
7
8  public:
9      FileWriter(const std::string& nom) : nomFichier(nom) {
10         fichier.open(nom);
11         if (!fichier.is_open()) {
12             throw std::runtime_error(
13                 "Impossible d'ouvrir le fichier : " + nom
14             );
15         }
16         std::cout << "Fichier " << nom << " ouvert en écriture\n";
17     }
18
19     ~FileWriter() {
20         if (fichier.is_open()) {
21             fichier.close();
22             std::cout << "Fichier " << nomFichier << " ferme\n";
23         }
24     }
25
26     void write(const std::string& ligne) {
27         fichier << ligne << std::endl;
28     }
29
30     // Interdit la copie
31     FileWriter(const FileWriter&) = delete;
32     FileWriter& operator=(const FileWriter&) = delete;
33 };
34

```

```
35 int main() {  
36     try {  
37         FileWriter fw("sortie.txt");  
38         fw.write("Premiere ligne");  
39         fw.write("Deuxieme ligne");  
40  
41         // Fichier ferme automatiquement ici  
42     } catch (const std::exception& e) {  
43         std::cerr << "Erreur : " << e.what() << "\n";  
44     }  
45 }
```

8 Pièges courants et bonnes pratiques

8.1 Piège 1 : Oublier le destructeur virtuel

Mauvais code

```

1 class Base {
2 public:
3     Base() { std::cout << "Base construite\n"; }
4     ~Base() { std::cout << "Base detruite\n"; } // PAS virtual !
5 };
6
7 class Derivee : public Base {
8 private:
9     int* data;
10 public:
11     Derivee() {
12         data = new int[100];
13         std::cout << "Derivee construite\n";
14     }
15     ~Derivee() {
16         delete[] data;
17         std::cout << "Derivee detruite\n";
18     }
19 };
20
21 int main() {
22     Base* ptr = new Derivee(); // Polymorphisme
23     delete ptr; // PROBLEME : seul ~Base() est appele !
24     // La memoire de Derivee fuite !
25 }

```

Solution

Toujours mettre virtual sur le destructeur de la classe de base !

```

1 class Base {
2 public:
3     virtual ~Base() { std::cout << "Base detruite\n"; }
4 };
5
6 class Derivee : public Base {
7     // ...
8     ~Derivee() override {
9         delete[] data;
10        std::cout << "Derivee detruite\n";
11    }
12 };
13
14 int main() {
15     Base* ptr = new Derivee();
16     delete ptr; // Appelle ~Derivee() puis ~Base() !
17 }

```

8.2 Piège 2 : La copie superficielle

Probleme

Si votre classe contient un pointeur, le constructeur de copie par défaut copie bêtement le pointeur. Resultat : deux objets pointent vers la meme memoire !

```
1 class MauvaiseClasse {
2     int* data;
3 public:
4     MauvaiseClasse(int taille) {
5         data = new int[taille];
6     }
7     ~MauvaiseClasse() {
8         delete[] data;
9     }
10    // Pas de constructeur de copie defini !
11 };
12
13 int main() {
14     MauvaiseClasse a(10);
15     MauvaiseClasse b = a; // Copie le pointeur
16 } // CRASH ! delete[] appele deux fois sur la meme memoire !
```

Solutions

Solution 1 : Interdire la copie

```
1 class NonCopiable {
2     int* data;
3 public:
4     NonCopiable(const NonCopiable&) = delete;
5     NonCopiable& operator=(const NonCopiable&) = delete;
6 };

```

Solution 2 : Implementer une copie profonde

```
1 class Copiable {
2     int* data;
3     size_t taille;
4 public:
5     Copiable(const Copiable& autre) {
6         taille = autre.taille;
7         data = new int[taille];
8         std::copy(autre.data, autre.data + taille, data);
9     }
10 };

```

8.3 Bonne pratique : La regle du zero ou la regle de cinq

Regle du zero

Si possible, n'crivez AUCUNE de ces fonctions :

- Destructeur
- Constructeur de copie
- Operateur d'affectation
- Move constructor
- Move assignment

Comment ? Utilisez des classes RAI existantes (std : :vector, std : :string, std : :unique_ptr, etc.)

```
1 class Bonne {  
2     std::vector<int> data; // Gere tout automatiquement !  
3     std::string nom;  
4     std::unique_ptr<Ressource> ressource;  
5     // Pas besoin d'ecrire de destructeur !  
6 };
```

Regle de cinq

Si vous devez ecrire l'une de ces fonctions, ecrivez LES CINQ :

- Destructeur
- Constructeur de copie
- Operateur d'affectation
- Move constructor
- Move assignment operator

Ou alors marquez les comme = delete si vous ne voulez pas qu'elles existent.

9 Quiz de validation

9.1 Questions

1. Qu'est-ce que RAI signifie et pourquoi est-ce important ?
2. Quelle est la difference entre `private` et `public` ?
3. Quand le destructeur d'un objet est-il appele ?
4. Pourquoi mettre `const` apres une methode ?
5. Que se passe-t-il si vous oubliez de liberer la memoire allouee avec `new` ?
6. Quelle est la difference entre passer un parametre par valeur et par reference ?
7. Pourquoi le constructeur de copie est-il important quand votre classe contient un pointeur ?
8. Dans quel ordre les destructeurs sont-ils appeles si vous avez plusieurs objets ?
9. Que signifie `= delete` apres une fonction ?
10. Citez trois ressources qui doivent etre gerees avec RAI.

9.2 Reponses

1. **RAI** = Resource Acquisition Is Initialization. Important car ca garantit que les ressources sont automatiquement liberees quand l'objet est detruit, meme en cas d'exception.
2. `private` : accessible seulement a l'interieur de la classe. `public` : accessible de partout. C'est l'encapsulation.
3. Automatiquement quand l'objet sort de la portee (accolades), ou quand `delete` est appele sur un pointeur.
4. Pour promettre que la methode ne modifie pas l'objet. Le compilateur verifie cette promesse.
5. Fuite memoire (memory leak). Le programme consomme de plus en plus de memoire jusqu'a crasher.
6. Par valeur : copie l'objet (lent). Par reference (&) : passe l'original sans copie (rapide).
7. Sans constructeur de copie personnalise, le compilateur copie betement les pointeurs. Deux objets pointent alors vers la meme memoire (dangereux!).
8. Dans l'ordre inverse de la construction (LIFO : dernier construit, premier detruit).
9. Interdit l'utilisation de cette fonction. Utile pour interdire la copie d'un objet.
10. Memoire (`new/delete`), fichiers (`fopen/fclose`), mutex (`lock/unlock`), connexions reseau, handles systeme, etc.

10 Pour aller plus loin

10.1 Exercice bonus : Smart Logger

Creez une classe `Logger` qui :

- Ecrit dans un fichier
- Ajoute automatiquement un timestamp a chaque message
- Compte le nombre de messages ecrits
- Affiche un resume dans le destructeur
- Gere les erreurs d'ouverture avec des exceptions

10.2 Exercice bonus : Mutex Guard

Creez une classe `MutexGuard` qui :

- Prend un mutex en parametre du constructeur
 - Verrouille le mutex dans le constructeur
 - Deverrouille automatiquement le mutex dans le destructeur
- Indice : C'est exactement ce que fait `std::lock_guard`!

10.3 Ressources supplementaires

- **cppreference.com** : Documentation complete sur les constructeurs, destructeurs, etc.
- **learncpp.com** : Tutoriel progressif sur les classes
- **Effective C++** de Scott Meyers : Le livre de reference (Item 13 sur RAI)

Fin de la Seance 1

Prochaine seance : Heritage et Polymorphisme